

Fundamentos de LLMs — Arquitectura Funcional

Taller 1 · Módulo 02 · Programa de Implementación de Agentes IA

Dr. Vicente Machaca Arceda | UTEC

Tabla de Contenidos

1. [Objetivos y Motivación](#)
 2. [Redes Neuronales: Bases Necesarias](#)
 3. [Definición de LLM](#)
 4. [Pipeline Completo de un LLM](#)
 5. [¿Cómo se Entrenaron los LLMs?](#)
 6. [Cuantización](#)
 7. [LLMs en la Industria](#)
 8. [Ventana de Contexto \(Context Window\)](#)
 9. [Function Calling](#)
 10. [Modelos Multi-modal](#)
 11. [Ejercicios Prácticos](#)
 12. [Conceptos Avanzados para Profundizar](#)
 13. [Referencias](#)
-

1. Objetivos y Motivación

Al finalizar este taller deberías poder:

- **Comprender** cómo los LLMs procesan texto, paso por paso, desde una cadena de caracteres hasta una predicción
- **Entender** qué son los embeddings y por qué son el corazón de la representación del conocimiento
- **Identificar** las limitaciones prácticas del contexto y cómo impactan al diseñar sistemas reales
- **Aplicar** conceptos de cuantización para elegir el modelo correcto según el hardware disponible
- **Usar** function calling para conectar LLMs con el mundo real

¿Por qué importa entender cómo funcionan internamente?

Muchos developers usan LLMs como cajas negras vía API. Eso funciona hasta que:

- El modelo "alucina" y no sabes por qué ni cómo reducirlo
- El costo escala de forma inesperada (no controlaste los tokens)
- El modelo "olvida" información importante de conversaciones largas
- Necesitas elegir entre GPT-5, Claude, Gemini o un modelo local — ¿cuál y por qué?

Entender la arquitectura te da las herramientas para tomar esas decisiones con criterio.

2. Redes Neuronales: Bases Necesarias

2.1 Parámetros: el "conocimiento" almacenado

Un modelo de lenguaje no "sabe" cosas de la manera en que los humanos sabemos. Su conocimiento está codificado como **millones de números** — los parámetros.

¿Qué son los parámetros?

- Son los **pesos y sesgos** de la red neuronal
- Definen cómo se transforman las entradas en salidas
- En un LLM moderno, van de millones hasta **cientos de miles de millones**
- Cada parámetro es un número real, normalmente en **float32** (32 bits = 4 bytes)

Tamaño en disco:

$$\text{Tamaño} \approx \text{parámetros} \times \frac{\text{bits por parámetro}}{8} \text{ bytes}$$

Un modelo de 7B parámetros en float32 ocupa ~28 GB. Por eso la cuantización (reducir a int8 o int4) es tan importante.

2.2 Entrenamiento: Descenso por Gradiente

Los parámetros no se programan manualmente — se **aprenden** a partir de datos.

Proceso:

1. Inicializar parámetros con valores aleatorios
2. Hacer una predicción con los datos actuales
3. Medir el error con una **función de pérdida** $L(\theta)$
4. Calcular el gradiente (dirección de mayor error)

5. Actualizar parámetros en dirección contraria al gradiente:

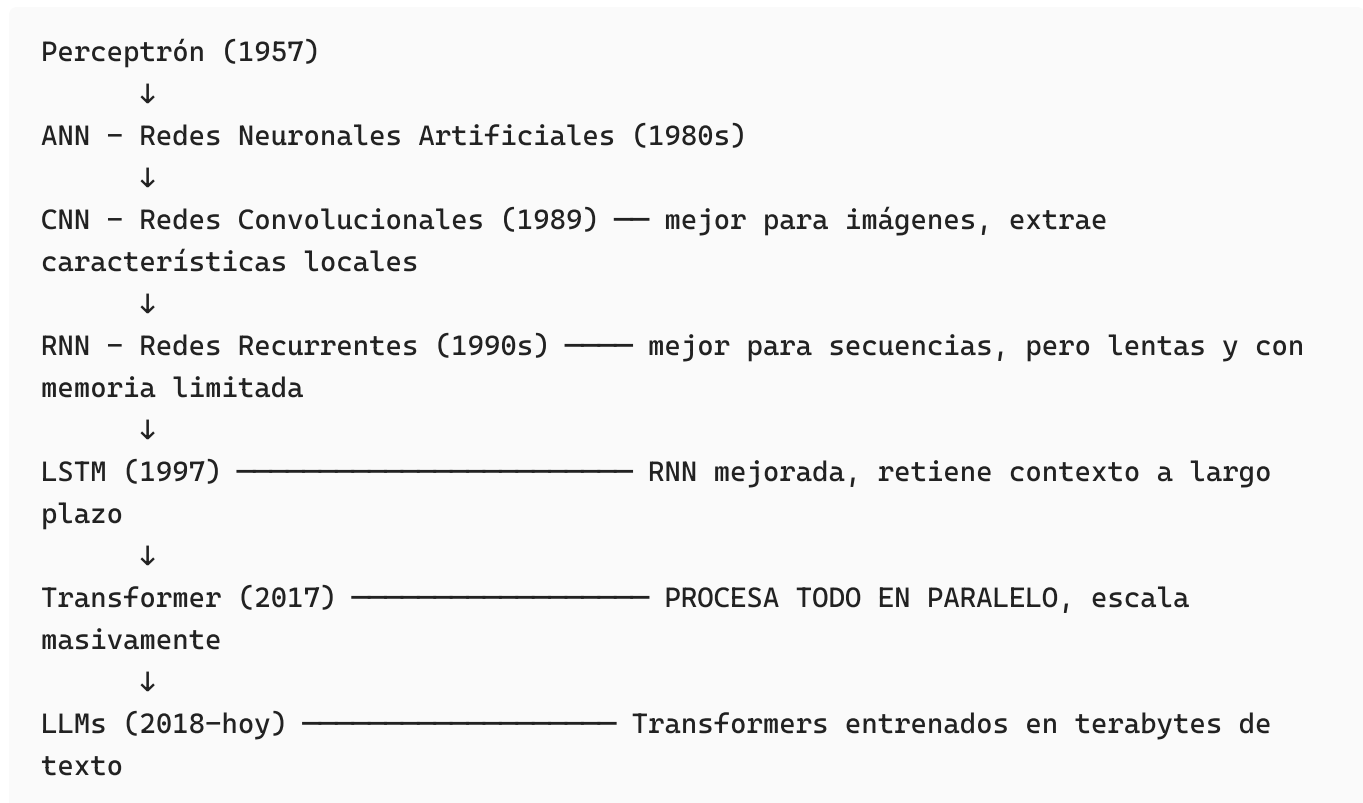
$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

Donde:

- θ = vector de todos los parámetros (pesos y sesgos)
- η = **learning rate** (tasa de aprendizaje) — qué tan grandes son los pasos
- $\nabla_{\theta} L(\theta)$ = gradiente de la pérdida respecto a los parámetros

Intuición: si un parámetro contribuyó al error, se ajusta un poco en la dirección que reduciría ese error. Con miles de millones de ejemplos, los parámetros convergen a valores que minimizan el error en general.

2.3 Evolución de las Arquitecturas



¿Por qué el Transformer superó a las RNNs?

Las RNNs procesan la secuencia **token por token** en serie. Para saber qué significa la palabra 50 en un texto, deben haber procesado las 49 anteriores secuencialmente. Esto tiene dos problemas:

1. **Lento:** no paralelizable → no escala bien con GPUs
2. **Memoria limitada:** el gradiente se desvanece al propagarse hacia atrás (problema del gradiente evanescente)

El Transformer **atiende a todos los tokens simultáneamente**, lo que lo hace masivamente paralelizable y capaz de capturar dependencias de largo alcance.

3. Definición de LLM

Aprendizaje Profundo (Deep Learning)

Subconjunto del Machine Learning que utiliza redes neuronales con **múltiples capas ocultas** para aprender representaciones jerárquicas de los datos.

Modelo de Lenguaje Grande (LLM)

Red neuronal profunda entrenada con **enormes corpus de texto** para comprender y generar lenguaje similar al humano.

Características que lo definen como "Grande":

Aspecto	Escala típica
Parámetros	7B – 1T+
Tokens de entrenamiento	1T – 15T
Cómputo de entrenamiento	10^{23} – 10^{25} FLOPs
Datos de entrenamiento	Terabytes de texto

Qué hace un LLM: dado un texto de entrada (contexto), predice la distribución de probabilidad sobre el vocabulario para el **siguiente token**. Nada más. La "inteligencia" emerge de repetir eso con suficientes parámetros y datos.

4. Pipeline Completo de un LLM

Cuando escribes "Los modelos son increíbles", esto es lo que ocurre internamente:

```
Texto crudo
  |
  ▼ Paso 1
[Tokenización] — "Los" | " modelos" | " son" | " incre" | "##íbles"
  |
  ▼ Paso 2
[IDs numéricos] — [1254, 8765, 201, 4599, 678]
  |
  ▼ Paso 3
```

```
[Embeddings] ——— vectores densos en  $R^d$  + codificación de posición
|
▼ Paso 4
[Capas Transformer] — N bloques de atención + feed-forward
|
▼
[Logits → Softmax] — distribución de probabilidad sobre ~50k tokens
|
▼
Siguiendo token predicho
```

Paso 1: Tokenización

Un **token** es la unidad mínima de procesamiento. No es necesariamente una palabra completa — puede ser una subpalabra, un carácter, o un símbolo.

¿Por qué no palabras completas?

- El vocabulario sería infinito (nombres propios, palabras nuevas, errores tipográficos)
- Código, matemáticas, URLs, emojis necesitan tratamiento especial
- Distintos idiomas tienen morfología muy diferente

BPE (Byte-Pair Encoding) — el más común (GPT-2, GPT-4, LLaMA):

El algoritmo aprende qué fragmentos de texto aparecen con más frecuencia y los convierte en tokens únicos:

```
"Los modelos son increíbles"
→ ["Los", " modelos", " son", " incre", "##íbles"]
```

Herramienta práctica: [TikTokenizer](#) — visualiza en tiempo real cómo GPT tokeniza cualquier texto.

Regla práctica:

- 1 token $\approx$ 0.75 palabras en inglés (~4 caracteres)
- En español: ~1.3-1.5 tokens/palabra (las tildes y morfología más rica generan más tokens)
- El código fuente es eficiente: palabras clave cortas, muchos caracteres por token

Implicación económica: las APIs cobran por token. Un prompt en inglés es más barato que el mismo en español o en idiomas con menos representación en el corpus.

Paso 2: De Tokens a IDs

El vocabulario es un diccionario donde cada token tiene un ID único:

```
["Los", " modelos", " son", " incre", "##íbles"]  
→ [1254, 8765, 201, 4599, 678 ]
```

GPT-2 tiene vocabulario de 50,257 tokens. GPT-4 usa ~100K tokens.

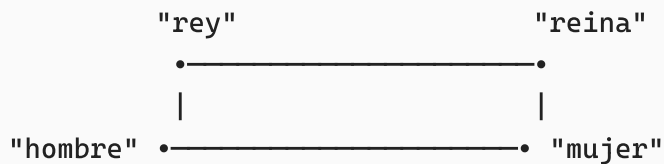
Paso 3: Embeddings

Cada ID numérico se convierte en un **vector denso** de alta dimensión.

¿Por qué vectores?

Porque los números enteros no capturan similitud semántica. Los **embeddings** resuelven esto: cada token se mapea a un punto en un espacio de, por ejemplo, 768 dimensiones, donde la distancia entre puntos refleja similitud semántica.

Espacio de embeddings (simplificado a 2D):



Propiedad fascinante:

```
vec("rey") - vec("hombre") + vec("mujer") ≈ vec("reina")  
vec("París") - vec("Francia") + vec("Italia") ≈ vec("Roma")
```

Las relaciones semánticas son operaciones vectoriales. El modelo no programó esto explícitamente — emergió del entrenamiento.

Los embeddings pueden crearse para distintos tipos de entrada:

- Texto (palabras, oraciones, documentos)
- Imágenes (parches de imagen)
- Audio (segmentos de audio)
- Código
- Esta unificación permite los modelos multi-modal

Codificación de posición (Positional Encoding):

Los embeddings por sí solos no saben en qué posición de la oración está cada token. "El gato persigue al ratón" y "El ratón persigue al gato" tienen los mismos tokens pero significados opuestos.

La codificación posicional añade información de posición al embedding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Modelos modernos (LLaMA, Mistral) usan **RoPE** (Rotary Position Embedding), más eficiente para contextos largos.

Paso 4: Procesamiento en el Transformer

El Transformer (Vaswani et al., 2017) es la arquitectura central de todos los LLMs modernos. Consiste en N bloques apilados, cada uno con dos componentes:

Mecanismo de Auto-Atención (Self-Attention)

Permite que cada token "mire" a todos los demás tokens de la secuencia y decida a cuáles prestar atención.

Intuición: en "El animal no cruzó la calle porque **estaba** cansado", ¿a qué se refiere "estaba"? ¿Al animal o a la calle? La atención permite al modelo aprender que "estaba" atiende fuertemente a "animal".

Matemáticamente — cada token genera tres vectores:

- **Q** (Query): "¿Qué estoy buscando?"
- **K** (Key): "¿Cómo me anuncio al resto?"
- **V** (Value): "¿Qué información comparto si me seleccionan?"

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention: múltiples "cabezas" en paralelo, cada una puede especializarse en relaciones distintas (sintácticas, semánticas, de coreferencia).

Red Feed-Forward (FFN)

Después de la atención, cada posición pasa por una red neuronal densa:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Investigaciones muestran que las FFNs almacenan **conocimiento factual** — son como memorias clave-valor. La atención "recupera" contexto; la FFN "aplica" conocimiento.

Paso 5: Predicción del Siguiente Token

$$P(\text{token} \mid \text{contexto}) = \text{softmax}(z)$$

Esto produce una **distribución de probabilidad** sobre los ~50,000 tokens del vocabulario:

```
Entrada: "Los modelos son"  
Probabilidades:  
  " increíbles" → 0.23  
  " buenos"      → 0.18  
  " útiles"      → 0.15  
  ...
```

El modelo elige un token, lo añade al contexto, y **repite el proceso**. Así se genera texto token a token.

5. ¿Cómo se Entrenaron los LLMs?

5.1 Pre-entrenamiento

El objetivo es predecir el siguiente token dado el contexto anterior (modelos GPT) o predecir tokens enmascarados (modelos BERT).

Datos: mezclas de texto de internet (Common Crawl), libros, Wikipedia, código de GitHub, artículos científicos — **terabytes** de texto preprocesado y deduplicado.

Costo: entrenar GPT-3 costó estimativamente \$4-12 millones en cómputo. Los modelos actuales cuestan decenas a cientos de millones de dólares.

5.2 Masked Language Modeling — BERT

BERT se entrenó enmascarando aleatoriamente el 15% de los tokens:

```
Original:   "El gato [MASK] sobre la [MASK]"  
Predicción: "El gato saltó sobre la mesa"
```

Esto permite un entrenamiento **bidireccional**: el modelo ve tanto contexto anterior como posterior, produciendo mejores representaciones para tareas de comprensión.

5.3 Fine-Tuning para Tareas Específicas

Después del pre-entrenamiento genérico, se ajusta el modelo para tareas concretas con un dataset más pequeño y específico:

```
Modelo pre-entrenado (conoce el lenguaje)
      ↓ Fine-tuning
Modelo especializado (sigue instrucciones / hace clasificación / genera código)
```

5.4 RLHF — Del modelo de texto al asistente

Para convertir un modelo que "completa texto" en un asistente útil se usa RLHF (Reinforcement Learning from Human Feedback):

1. **SFT** (Supervised Fine-Tuning): demostrar al modelo cómo responder bien
2. **Reward Model**: humanos comparan respuestas; se entrena un modelo para predecir preferencias
3. **PPO**: el LLM aprende a generar respuestas que el Reward Model califica bien

Resultado: un asistente que es **útil, inofensivo y honesto**.

6. Cuantización

¿Qué es y por qué importa?

Cuando descargues un LLM local (Ollama, LM Studio), verás nombres como `Mistral-7B-Instruct-Q4_K_M.gguf`. La cuantización es la técnica detrás de eso.

Definición: reducir la precisión numérica de los parámetros para disminuir el tamaño del modelo y acelerar la inferencia, con mínima pérdida de calidad.

El problema del tamaño:

```
Llama 3 70B en float32:
  70,000,000,000 params × 4 bytes = 280 GB
  → Necesitarías 4× NVIDIA A100 (80GB cada una)
```

```
Llama 3 70B en Q4:
  70,000,000,000 params × 0.5 bytes ≈ 35 GB
  → Cabe en una PC gaming con 2 GPUs RTX 4090
```

Niveles de Cuantización

Nivel	Bits/parámetro	Tamaño relativo	Calidad
float32	32 bits	100%	Referencia
float16	16 bits	50%	Casi idéntica
Q8	8 bits	25%	Casi sin pérdida
Q5	5 bits	~16%	Muy buena
Q4	4 bits	12.5%	Buena — balance ideal
Q2	2 bits	6.25%	Muy ligero, baja precisión

Nomenclatura de Modelos Cuantizados

Ejemplo: `Mistral-7B-Instruct-Q4_K_M.gguf`

Parte	Significado
Mistral	Familia del modelo
7B	7 mil millones de parámetros
Instruct	Fine-tuned para seguir instrucciones
Q4	Cuantización a 4 bits por parámetro
K	Grouped quantization (asigna más bits a capas críticas)
M	Medium — optimización de memoria balanceada
.gguf	Formato optimizado para llama.cpp / ejecución local

Sufijos comunes:

- `_0` : básico, rápido pero menos preciso
- `_1` : esquema alternativo, más calidad
- `_K` : grouped quantization
- `_M` : optimización de memoria

Ventajas y Desventajas

Ventajas:

- Reducción de memoria hasta **75%** o más
- Inferencia más rápida en CPU/GPUs pequeñas

- Permite ejecutar LLMs en **laptops** o incluso móviles
- Acceso democrático sin depender de cloud

Desventajas:

- Pérdida ligera de precisión en la predicción
- No todas las arquitecturas toleran bien cuantización extrema (Q2 puede degradar bastante)
- Aritmética y razonamiento lógico se ven más afectados que generación de texto fluido

Herramientas para LLMs Locales

[Ollama](#) — CLI para correr LLMs localmente:

```
# TinyLlama: 638MB, contexto 2K, perfecto para pruebas
ollama run tinyllama

# Mistral 7B Q4
ollama run mistral
```

[LM Studio](#) — interfaz gráfica para descargar y chatear con modelos. Busca TinyLlama/TinyLlama-1.1B-Chat-v0.6 como primer modelo de prueba.

7. LLMs en la Industria

Timeline de Modelos Clave

2018	GPT-1 (OpenAI)	—————	117M params, primer GPT
2018	BERT (Google)	—————	bidireccional, domina NLP
2019	GPT-2 (1.5B)	—————	"demasiado peligroso para publicar"
2020	GPT-3 (175B)	—————	few-shot learning, la revolución
2021	Codex	—————	→ GitHub Copilot
2022	ChatGPT	—————	RLHF, el primer asistente masivo
2023	GPT-4, LLaMA, Mistral	—	modelos open source despegan
2024	Gemini 1.5 (1M ctx)	—	contextos masivos
2024	LLaMA 3.1 405B	—————	mejor open source
2025	Claude Fable 5, GPT-5	—	nueva generación
2026	Llama 4 Scout (10M ctx), Kimi K2.6 (1T MoE)		

Modelos Proprietarios Actuales (junio 2026)

LLM	Context Window	Max Output
Gemini 3.5 Pro	2,000,000	65,536
Gemini 3.5 Flash	1,048,576	65,536
GPT-5.5	1,050,000	128,000
GPT-5.4	1,050,000	128,000
Claude Fable 5	1,000,000	128,000
Claude Opus 4.8	1,000,000	128,000
Claude Sonnet 4.6	1,000,000	64,000

En Gemini, el context window es el límite de tokens de **entrada** (la salida es adicional). En OpenAI y Anthropic es el presupuesto **total** compartido entre entrada y salida.

Modelos Open Source/Open Weights (junio 2026)

LLM	Parámetros	Context Window	Max Output
Kimi K2.6	1T (MoE)	256,000	8,192
DeepSeek V4	671B (MoE)	1,000,000	64,000
DeepSeek V3	671B (MoE)	128,000	8,192
Llama 4 Maverick	400B (MoE)	1,000,000	8,192
Llama 4 Scout	109B (MoE)	10,000,000	8,192
Qwen 3.5	235B (MoE)	256,000	8,192
Mistral Medium 3.5	—	128,000	8,192
Gemma 4	27B	128,000	8,192

MoE = Mixture of Experts: el modelo tiene muchos parámetros en total, pero solo activa un subconjunto por token. DeepSeek V4 tiene 671B parámetros totales pero solo activa ~37B por token — más conocimiento con costo computacional de un modelo pequeño.

Benchmarks

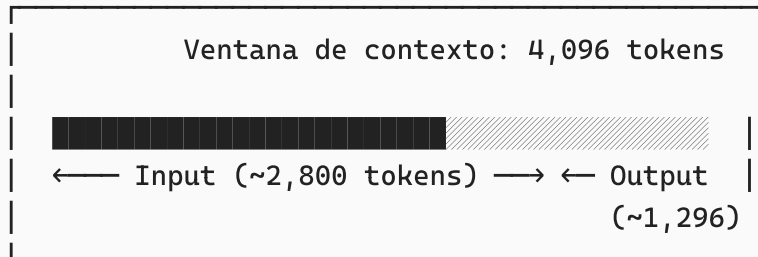
- vllm.ai/llm-leaderboard — comparativa de capacidades y precios
 - livebench.ai — benchmark dinámico con preguntas nuevas para evitar contaminación de datos
-

8. Ventana de Contexto (Context Window)

Este es uno de los conceptos más importantes para el diseño de sistemas reales.

Definición

La **ventana de contexto** es la cantidad máxima de tokens (entrada + salida) que un modelo puede procesar simultáneamente.



Max Token Input vs Max Token Output

Max Token Input: máximo de tokens que puedes enviar como entrada.

```
Ventana total = 4,096 tokens
Input = 3,000 tokens
→ Output máximo posible = 1,096 tokens
```

Max Token Output: máximo de tokens que el modelo puede generar.

```
Ventana total = 2,048 tokens
Input = 500 tokens
→ Output máximo = 1,548 tokens
```

Si el input llena casi toda la ventana, el output queda truncado. Las "respuestas cortadas" suelen deberse a esto.

Limitaciones Prácticas

Truncamiento: si el texto de entrada excede la ventana, el modelo ignora tokens desde el inicio.

Pérdida de contexto en diálogos largos: en una conversación larga, los mensajes más antiguos quedan fuera de la ventana. El modelo **olvida** lo que se dijo al inicio.

Costo computacional: la atención tiene complejidad $O(n^2)$ en la longitud de secuencia. Ventanas más grandes requieren exponencialmente más memoria y cómputo.

Referencia práctica:

```
1,000 tokens  ≈  750 palabras  ≈  1.5 páginas
4,096 tokens  ≈  3,000 palabras ≈  ~6 páginas
                Un PDF de 20 páginas NO cabe completo
128K tokens   ≈  ~250 páginas (una novela corta)
1M tokens     ≈  ~2,000 páginas
```

Anatomía de un Fallo de Contexto

El Dr. Machaca propone este experimento con TinyLlama (contexto de solo 2K tokens):

```
1. Establecer prioridad
   User: "ProjectAlpha has the major priority."
   IA:   "OK. Alpha is priority."

2. Confirmar
   User: "Which project has the major priority?"
   IA:   "Alpha..."

3. Llenar el contexto
   [Se discuten 10+ proyectos más con muchos detalles]
   [La información de Alpha se desplaza fuera de la ventana]

4. El Fallo
   User: "¿Cuál era el proyecto prioritario?"
   IA:   "No mencionaste un proyecto prioritario."
```

Lección: el LLM no "olvidó" por distracción — la información simplemente **ya no está en su ventana de proceso**. No hay retención entre conversaciones ni memoria implícita.

Soluciones a las Limitaciones de Contexto

RAG — Retrieval-Augmented Generation

En lugar de meter todos los documentos en el contexto, se indexan en una base de datos vectorial y se recuperan solo los fragmentos relevantes:

```
Pregunta del usuario
      ↓
[Embedding de la pregunta]
```

↓
[Búsqueda en Vector DB] → chunks más similares semánticamente
↓
[Chunks recuperados + Pregunta] → LLM
↓
Respuesta basada en los documentos recuperados

Un PDF de 20 páginas no cabe en 4,096 tokens, pero con RAG puedes indexarlo completo y recuperar solo las partes relevantes a cada pregunta.

Memoria en Agentes

Los sistemas multi-agente mantienen un componente de memoria separado (base de datos, archivos) que persiste entre conversaciones. El agente decide qué guardar y cuándo recuperar.

9. Function Calling

¿Qué es?

Function Calling permite que los LLMs se conecten a herramientas externas y al mundo real.

Punto clave: el LLM **no ejecuta** la función — genera una salida estructurada (JSON) indicando qué función llamar y con qué argumentos. El **desarrollador** recibe ese JSON y ejecuta la función en su código.

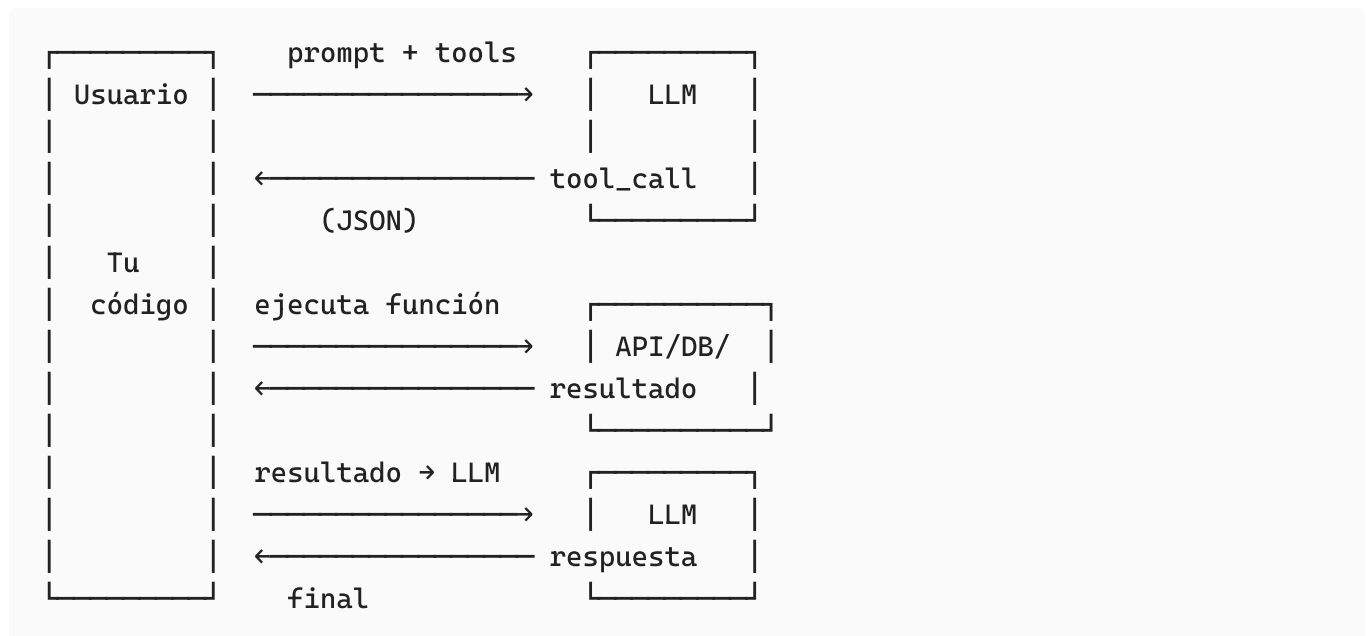
```
Usuario: "¿Cuánto está AAPL ahora?"  
↓  
LLM analiza: necesito información de precios en tiempo real  
↓  
LLM genera JSON:  
{  
  "function": "get_stock_price",  
  "arguments": {"symbol": "AAPL"}  
}  
↓  
Tu código ejecuta: api.get_stock_price("AAPL") → {"price": 189.50}  
↓  
LLM recibe el resultado y responde al usuario:  
"Apple (AAPL) cotiza actualmente en $189.50"
```

¿Por qué es tan importante?

Sin Function Calling, los LLMs están limitados a su conocimiento estático. Con Function Calling, pueden:

- Consultar **datos en tiempo real** (precios, clima, noticias)
- **Ejecutar cálculos** precisos en Python/código
- **Buscar en bases de datos** internas de la empresa
- **Interactuar con APIs** (enviar emails, crear eventos, actualizar CRMs)
- **Controlar sistemas** (automatización, DevOps, domótica)

Diagrama de Flujo



Ejemplo con Gemini API (Python)

```
import google.generativeai as genai

tools = [{
    "function_declarations": [{
        "name": "get_weather",
        "description": "Obtiene el clima actual de una ciudad",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "Nombre de la ciudad"}
            }
        },
        "required": ["city"]
    }]
}]
```

```
model = genai.GenerativeModel("gemini-1.5-pro", tools=tools)
response = model.generate_content("¿Cómo está el clima en Lima?")

# El LLM decide cuándo llamar la función
if response.candidates[0].content.parts[0].function_call:
    fc = response.candidates[0].content.parts[0].function_call
    # → function: get_weather, args: {'city': 'Lima'}
```

Terminología por proveedor:

- **OpenAI**: "Function Calling" / "Tool Use"
- **Anthropic (Claude)**: "Tool Use"
- **Google (Gemini)**: "Function Calling"

Son el mismo concepto con distintos nombres.

10. Modelos Multi-modal

Los LLMs ya no se limitan a texto. Los modelos multi-modal procesan y generan múltiples tipos de datos:

```
Texto + Imágenes → LLM → Texto (GPT-4o, Claude 3.5, Gemini)
Texto + Audio    → LLM → Texto (Whisper + GPT)
Texto            → LLM → Imágenes (DALL-E 3)
Texto + Video    → LLM → Texto (Gemini 1.5)
```

Cómo funciona: las imágenes se dividen en "parches" (patches), cada uno se convierte en un embedding, y esos embeddings se concatenan con los embeddings del texto. El Transformer procesa todo en un espacio vectorial unificado.

Casos de uso:

- "Describe esta imagen" / "¿Qué error muestra esta captura de pantalla?"
 - "Analiza este gráfico de ventas y dame conclusiones"
 - "Lee el texto en esta foto de un documento"
 - "Encuentra errores en este diagrama de arquitectura"
 - Generación de imágenes a partir de descripciones textuales
-

11. Ejercicios Prácticos

Ejercicio 1: LLMs y Aritmética de Alta Precisión

Objetivo: entender por qué los LLMs fallan en cálculos precisos.

Pregunta a **ChatGPT**, **Gemini** y **Grok** (en chats distintos):

¿Cuánto es $0.234523452345234523 \times 0.5687657856785887851$?

Verifica con Python:

```
from decimal import Decimal, getcontext
getcontext().prec = 40
a = Decimal("0.234523452345234523")
b = Decimal("0.5687657856785887851")
print(a * b)
```

Análisis: los LLMs no hacen aritmética real — predicen el **texto más probable** como respuesta. Los números de alta precisión raramente aparecen en internet, y la multiplicación larga requiere "recordar" carries intermedios — algo para lo que el Transformer no tiene mecanismo dedicado.

Moraleja: para cálculos precisos, usar **Function Calling** para delegar a Python.

Ejercicio 2: Idiomas y Tokenización

Objetivo: observar cómo distintos idiomas consumen diferentes cantidades de tokens.

1. Escribe "hi" al LLM
2. Traduce "hi" a **Shan** (idioma birmano) en Google Translate
3. Escribe el saludo en Shan al LLM
4. Observa la diferencia en las respuestas

Verifica en [TikTokenizer](#):

- "hi" = 1 token
- El equivalente en Shan = múltiples tokens

¿**Por qué importa?** Un prompt en un idioma poco frecuente en el corpus de entrenamiento:

- Consume más tokens (más caro)
- Tiene peor comprensión por parte del modelo

- Puede producir respuestas mezcladas con otros idiomas

Ejercicio 3: Anatomía de un Fallo de Contexto

Objetivo: observar el olvido de contexto con TinyLlama.

1. Instala [Ollama](#) y ejecuta: `ollama run tinyllama`
2. Establece: "ProjectAlpha has the major priority"
3. Confirma: "Which project has the major priority?" → debe responder bien
4. Discute 10-15 proyectos ficticios con muchos detalles (llena el contexto)
5. Pregunta de nuevo: "Which project has the major priority?"
6. Observa cómo el modelo ya no recuerda

Alternativa en LM Studio: descarga `TinyLlama/TinyLlama-1.1B-Chat-v0.6`.

Ejercicio 4: Explorar Benchmarks

Visita [livebench.ai](#) y analiza:

- ¿En qué tareas hay mayor diferencia entre modelos grandes y pequeños?
- ¿Qué tan cerca están los mejores modelos open source de los propietarios?
- ¿Qué tipo de tareas son más difíciles para los LLMs en general?

12. Conceptos Avanzados para Profundizar

KV Cache

Cuando generas el token 100, el modelo necesitaría recalcular K y V para todos los tokens anteriores. El **KV Cache** guarda estos valores:

- Sin cache: $O(n^2)$ operaciones por token generado
- Con cache: $O(n)$ — solo calcular para el nuevo token

Por eso el **primer token** (TTFT — Time to First Token) tarda más que los siguientes. En sistemas productivos, el KV Cache es fundamental para rendimiento.

Sampling: Temperatura y Nucleus

Temperature:

$$P'(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T = 0$: determinista (siempre el más probable) → respuestas consistentes pero repetitivas
- $T = 1$: distribución original
- $T > 1$: más aleatorio/creativo → puede volverse incoherente

Top-p (Nucleus Sampling): elegir del conjunto mínimo de tokens que sumen probabilidad p :

`top_p = 0.9` → tomar suficientes tokens para acumular 90% de probabilidad

Si la distribución es concentrada, bastan 2-3 tokens. Si es plana, puede incluir cientos.

Configuración típica: `temperature=0.7`, `top_p=0.95` para un asistente conversacional.

Prompt Engineering: Técnicas Clave

Técnica	Descripción	Cuándo usar
Zero-shot	Solo la instrucción, sin ejemplos	Tareas simples, el modelo ya sabe
Few-shot	Instrucción + ejemplos	Formato específico o comportamiento no estándar
Chain-of-Thought	"Pensemos paso a paso..."	Razonamiento complejo, matemáticas
Role Prompting	"Eres un experto en..."	Tono específico o expertise de dominio
Structured Output	"Responde en JSON con campos..."	Integración con código

Chain-of-Thought en acción:

- ❌ Sin CoT:
 Pregunta: ¿15% de 340?
 Respuesta: 51 (puede fallar en cálculos complejos)
- ✅ Con CoT:
 Pregunta: ¿15% de 340? Piensa paso a paso.
 Respuesta:
 1. 10% de 340 = 34
 2. 5% = mitad de 10% = 17
 3. 15% = 34 + 17 = 51

Capacidades Emergentes

Habilidades que aparecen **abruptamente** en modelos grandes y no estaban presentes en modelos pequeños:

- **Few-shot learning**: adaptar el modelo a nuevas tareas con solo ejemplos en el prompt
- **Chain-of-Thought**: razonar paso a paso (emerge alrededor de los 100B parámetros)
- **Aritmética básica**: resolver operaciones simples
- **Traducción sin datos de traducción**: modelos entrenados en inglés pueden traducir a otros idiomas

Las capacidades emergentes hacen difícil predecir qué puede hacer un modelo más grande — tanto en capacidades como en riesgos.

Alucinaciones: El Problema Central

Los LLMs **inventan información con confianza**. Esto no es un bug — es una consecuencia de cómo funcionan.

¿Por qué ocurre?

- El modelo optimiza para texto fluido y coherente, no para veracidad
- La distribución de probabilidad siempre produce algo — nunca "no sé"
- "Conocer" un hecho vs "generar texto plausible sobre ese hecho" no es lo mismo

Mitigaciones:

- **RAG**: anclar al modelo en documentos reales y verificables
- **Calibración**: enseñar al modelo a expresar incertidumbre ("No estoy seguro de...")
- **Verificación**: comparar las respuestas con fuentes externas
- **Function Calling**: para datos específicos, consultar fuentes reales

13. Referencias

Referencias del Curso

1. Yann LeCun, Yoshua Bengio, Geoffrey Hinton — "Deep learning" — *Nature*, 2015
2. Tom Brown et al. — "Language Models are Few-Shot Learners" (GPT-3) — *NeurIPS*, 2020
3. Jay Alammar & Maarten Grootendorst — **Hands-on Large Language Models** — O'Reilly, 2024
4. Ashish Vaswani et al. — "Attention Is All You Need" — *NeurIPS*, 2017

Recursos Online Recomendados

- [The Illustrated Transformer](#) — visualización más clara que existe del Transformer
- [Andrej Karpathy — Let's build GPT](#) — implementar GPT desde cero en ~1h de video
- [TikTokenizer](#) — explorar tokenización en tiempo real
- [LLM Visualization](#) — visualización 3D interactiva del forward pass
- [HuggingFace Course](#) — curso práctico gratuito
- [Lil'Log](#) — artículos técnicos profundos sobre LLMs

Herramientas del Taller

- [Ollama](#) — modelos locales vía CLI
- [LM Studio](#) — interfaz gráfica para modelos locales
- [TinyLlama en Ollama](#) — modelo de prueba (638MB, 2K contexto)
- [vellum.ai/llm-leaderboard](#) — comparativa de modelos
- [livebench.ai](#) — benchmark dinámico

Dr. Vicente Machaca Arceda — `vmachaca@utec.edu.pe`

[LinkedIn](#)

Este documento integra los contenidos del Taller 1 con expansiones conceptuales para construir una comprensión sólida y aplicable de los fundamentos de LLMs.